Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 3
по дисциплине «Анализ алгоритмов»

Тема Вычисление процессорного времени реализаций различных алгоритмов умножения матриц.

Студент Одинцов Е.В.

Группа ИУ7-63БВ

Преподаватели Волкова Л.Л.

Москва, 2025

Содержание

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

2 Технологическая часть **6**

2.1 Файл сборки ПО 6

2.2 Реализация стандартного алгоритма умножения матриц 6

2.3 Базовая реализация алгоритма Винограда 7

2.4 Реализация алгоритма Винограда с оптимизациями 8

3 Исследовательская часть **11**

3.1 Методология замера времени работы реализаций алгоритмов умножения матриц 11

3.2 Результат замеров 12

3.2.1 Время выполнения реализаций алгоритмов 13

3.2.2 Отношение производительности 14

3.2.3 Анализ результатов 14

ЗАКЛЮЧЕНИЕ **15**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **16**

ВВЕДЕНИЕ

Целью данной работы является реализация и исследование эффективности различных алгоритмов умножения матриц: стандартного алгоритма, алгоритма Винограда и его оптимизированной версии. В процессе работы также проводится измерение процессорного времени выполнения реализации каждого алгоритма для оценки его производительности в зависимости от размера матриц.

1 Аналитическая часть

Умножение матриц является одной из базовых операций в линейной алгебре и находит широкое применение в компьютерных науках, инженерии, физике и других областях.

Стандартный алгоритм умножения матриц имеет кубическую временную сложность $O(n^3)$. Существует ряд алгоритмов той же сложности с меньшим коэффициентом при старшем члене в формуле трудоемкости, что позволяет повысить временную эффективность расчета при больших размерах матриц. Для ускорения расчётов используется алгоритм Винограда, уменьшающий количество операций умножения за счёт предварительных вычислений дополнительных коэффициентов.

Алгоритм Винограда особенно эффективен при работе с большими матрицами. Однако его базовая версия может быть дополнительно оптимизирована с целью повышения производительности за счёт упрощения выражений и сокращения количества обращений к памяти.

2 Технологическая часть

В качестве языка программирования выбран C++ стандарта C++23. Для управления сборкой проекта используется CMake версии 3.31.

Основные компоненты программы следующие.

— Файл сборки программы CMakeLists.txt (см. листинг 2.1).

— Реализация трёх алгоритмов умножения матриц:

  — стандартный алгоритм (см. листинг 2.2);

  — базовый алгоритм Винограда (см. листинг 2.3);

  — оптимизированный алгоритм Винограда (см. листинг 2.4).

— Генерация случайных матриц заданного размера.

— Измерение времени выполнения алгоритмов.

— Вывод результатов в консоль и сохранение в CSV-файлы.

2.1 Файл сборки ПО

В листинге 2.1 представлен файл сборки, свидетельствующий о том, что оптимизация компилятора отключена.

Листинг 2.1 — CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(matrices)

set(CMAKE_CXX_STANDARD 23)

set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -O0") //без оптимизаций

add_executable(matrices main.cpp)
```

2.2 Реализация стандартного алгоритма умножения матриц

В листинге 2.2 представлен код реализации стандартного алгоритма умножения матриц.

Листинг 2.2 — стандартное умножение матриц

```
Matrix multiply(const Matrix &A, const Matrix &B) {
  if (A[0].size() != B.size()) throw invalid_argument("
   Incompatible matrix dimensions");
```

```
  size_t n = A.size();
  size_t m = B.size();
  size_t p = B[0].size();
  Matrix result(n, vector<int>(p, 0));

  for (size_t i = 0; i < n; ++i)
  for (size_t j = 0; j < p; ++j)
  for (size_t k = 0; k < m; ++k)
  result[i][j] = result[i][j] + A[i][k] * B[k][j];

  return result;
}
```

2.3 Базовая реализация алгоритма Винограда

В листинге 2.3 представлен код базовой реализации алгоритма Винограда.

Листинг 2.3 — Базовая реализация алгоритма Винограда

```
Matrix vinogradBasic(const Matrix &A, const Matrix &B) {
  if (A[0].size() != B.size()) throw invalid_argument("
   Incompatible matrix dimensions");

  size_t n = A.size();
  size_t m = A[0].size();
  size_t p = B[0].size();
  Matrix C(n, vector<int>(p, 0));

  vector<int> rowFactor(n, 0);
  vector<int> colFactor(p, 0);

  for (size_t i = 0; i < n; ++i)
  for (size_t j = 0; j < m / 2; ++j)
  rowFactor[i] = rowFactor[i] + A[i][2 * j] * A[i][2 * j + 1];

  for (size_t j = 0; j < p; ++j)
  for (size_t i = 0; i < m / 2; ++i)
  colFactor[j] = colFactor[j] + B[2 * i][j] * B[2 * i + 1][j];

  for (size_t i = 0; i < n; ++i) {
```

```
    for (size_t j = 0; j < p; ++j) {
      C[i][j] = -rowFactor[i] - colFactor[j];
      for (size_t k = 0; k < m / 2; ++k) {
        C[i][j] = C[i][j] + (A[i][2 * k] + B[2 * k + 1][j]) * (A[
   i][2 * k + 1] + B[2 * k][j]);
      }
    }
  }

  if (m % 2 != 0) {
    for (size_t i = 0; i < n; ++i)
    for (size_t j = 0; j < p; ++j)
    C[i][j] = C[i][j] + A[i][m - 1] * B[m - 1][j];
  }

  return C;
}
```

2.4 Реализация алгоритма Винограда с оптимизациями

В листинге 2.4 представлен код реализации алгоритма Винограда с оптимизациями согласно варианту. Использоапны следующие оптимизации:

— объединены III и IV части алгоритма, что позволило избавиться от совпадающей пары циклов;

— заменно умножение на два на двоичный сдвиг целого числа;

— использовано кэширование рассчитываемых величин;

— использование += в теле цикла.

Листинг 2.4 — Реализация алгоритма Винограда с оптимизациями

```
Matrix vinogradOptimized(const Matrix &A, const Matrix &B) {
  size_t n = A.size();
  size_t m = A[0].size();
  size_t p = B[0].size();

  if (m != B.size()) throw invalid_argument("Incompatible matrix
   dimensions");

  Matrix C(n, vector<int>(p, 0));
  size_t halfM = (m >> 1);
```

```
  bool isOdd = (m & 1);
  size_t lastIdx = (m - 1);

  vector<int> rowFactor(n, 0);
  vector<int> colFactor(p, 0);

  for (size_t i = 0; i < n; ++i) {
    int sum = 0;
    for (size_t j = 0; j < halfM; ++j) {
      size_t idx = j << 1;
      sum += A[i][idx] * A[i][idx + 1];
    }
    rowFactor[i] = sum;
  }

  for (size_t j = 0; j < p; ++j) {
    int sum = 0;
    for (size_t i = 0; i < halfM; ++i) {
      size_t idx = i << 1;
      sum += B[idx][j] * B[idx + 1][j];
    }
    colFactor[j] = sum;
  }

  for (size_t i = 0; i < n; ++i) {
    for (size_t j = 0; j < p; ++j) {
      int sum = -rowFactor[i] - colFactor[j];
      for (size_t k = 0; k < halfM; ++k) {
        size_t idx = k << 1;
        sum += (A[i][idx] + B[idx + 1][j]) * (A[i][idx + 1] + B[
 idx][j]);
      }
      if (isOdd) {
        sum += A[i][lastIdx] * B[lastIdx][j];
      }
      C[i][j] = sum;
    }
  }

  return C;
```

```
49 }
```

3 Исследовательская часть

3.1 Методология замера времени работы реализаций алгоритмов умножения матриц

Для сравнения эффективности различных реализаций алгоритмов умножения матриц был проведен замер процессорного времени реализаций алгоритмов умножения на матрицах различной размерности. Для каждой пары матриц было измерено среднее время выполнения каждой реализации. Измерения проводились с использованием высокоточного таймера Windows (функция `QueryPerformanceCounter` [6]).

Для корректности замеров выполнялось следующее.

— Для замера времени генерировалась пара матриц одинаковой размерности. Размерность варьировалась.

— Каждая реализация алгоритма запускалась для каждого входного значения по 5 раз, результат замера процессорного времени усреднялся.

— Замеры выполнялись на квадратных матрицах с четной линейной размерностью 300, 400, 500, 600, 700 для лучшего случая и на квадратных матрицах с нечетной линейной размерностью 301, 401, 501, 601, 701 для худшего случая.

— Проверялась корректность результатов работы реализаций всех алгоритмов.

Результаты измерений времени выполнения (среднее за 5 запусков) приведены ниже.

Лучший случай

Таблица 3.1 — Время выполнения реализаций алгоритмов при чётных размерах матриц

Размер	Стандартный алгоритм (с)	Виноград (базовый) (с)	Виноград (оптимизированный) (с)
300	0.198541	0.156563	0.106116
400	0.465650	0.367231	0.255863
500	0.925555	0.740616	0.510335
600	1.610350	1.295770	0.894591
700	2.658250	2.130640	1.472080

Худший случай

Таблица 3.2 — Время выполнения реализаций алгоритмов при нечётных размерах матриц

Размер	Стандартный алгоритм (с)	Виноград (базовый) (с)	Виноград (оптимизированный) (с)
301	0.216063	0.170456	0.115265
401	0.497931	0.396375	0.283573
501	0.992693	0.785887	0.534278
601	1.680370	1.351570	0.966161
701	2.892410	2.416820	1.510720

3.2 Результат замеров

На рисунке 3.1 представлено сравнение времени выполнения различных реализаций алгоритмов умножения матриц. Графики отображают результаты для двух случаев: наилучшего (Best Case) и худшего (Worst Case) на основе случайно сгенерированных матриц.

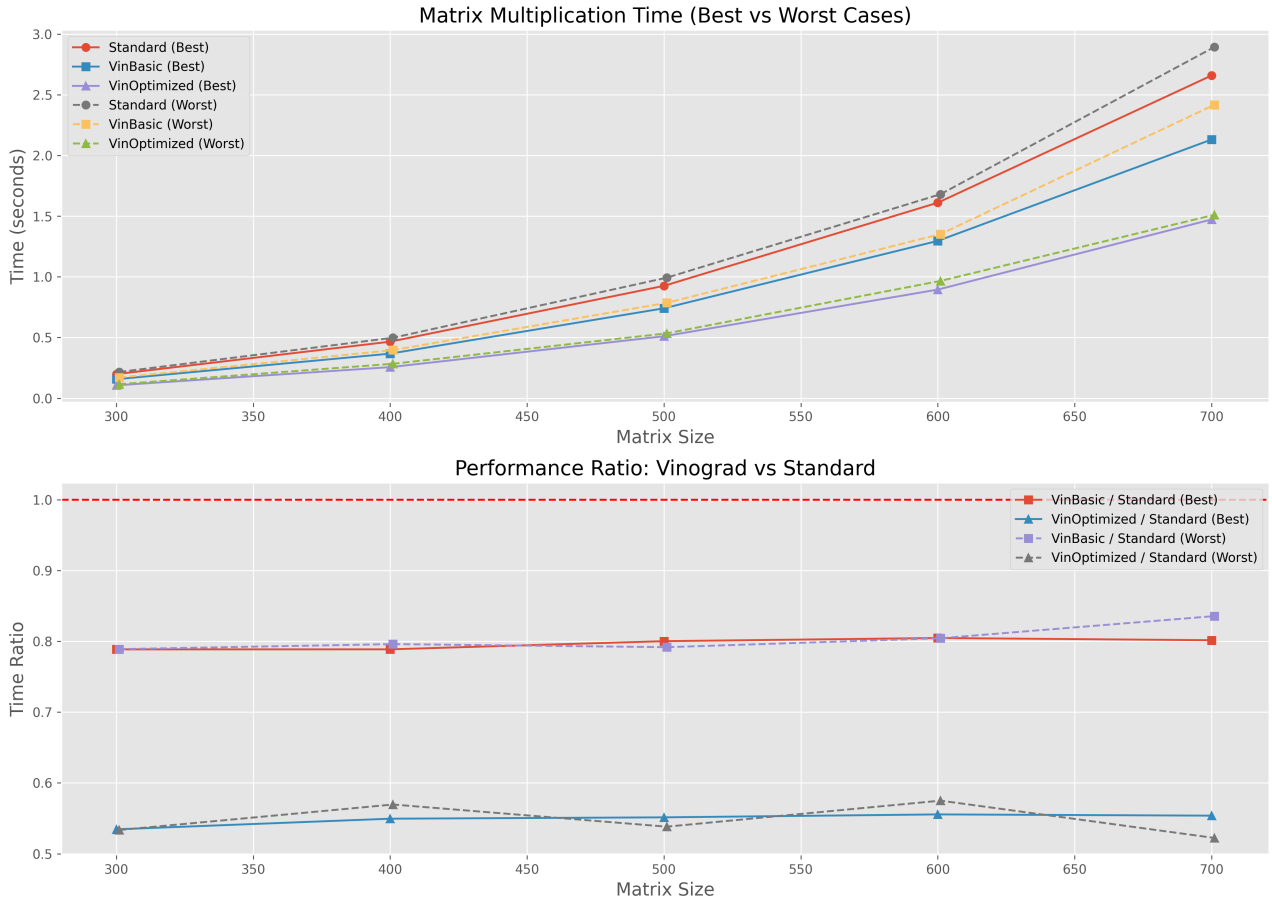


Рисунок 3.1 — Сравнение времени выполнения различных реализаций алгоритмов умножения матриц. На верхнем графике показано время выполнения стандартного алгоритма и алгоритмов Винограда в наилучшем и худшем случаях для матриц различных размеров. На нижнем графике показано отношение производительности алгоритмов Винограда к стандартному алгоритму, где значения меньше 1 означают, что алгоритм Винограда работает быстрее.

3.2.1 Время выполнения реализаций алгоритмов

В верхней части на рис. 3.1 показано время выполнения реализаций стандартного алгоритма (Standard) умножения матриц и двух вариантов алгоритма Винограда — базового (VinBasic) и оптимизированного (VinOptimized). Время выполнения измеряется в секундах.

Графики с маркерами обозначают следующие алгоритмы:

— Стандартный алгоритм умножения матриц (Standard).

— Базовая версия алгоритма Винограда (VinBasic).

— Оптимизированная версия алгоритма Винограда (VinOptimized).

Результаты представлены для наилучших и худших случаев. Непрерыв-

ные линии соответствуют наилучшему случаю (Best), а штриховые линии — худшему (Worst).

3.2.2 Отношение производительности

На нижней части на рис. 3.1 отображено отношение производительности алгоритмов Винограда к стандартному алгоритму умножения матриц. Отношение производительности вычисляется как время работы реализации алгоритма Винограда, деленное на время работы реализации стандартного алгоритма. Пунктирной линией без маркеров отмечен уровень единицы, соответствующий стандартному алгоритму.

Графики показывают, как меняется производительность реализации алгоритмов Винограда в зависимости от линейного размера матрицы. Если значение отношения меньше 1, это означает, что соответствующий алгоритм Винограда работает быстрее, чем стандартный.

Графики для наилучшего и худшего случаев также представлены, где непрерывные и прерывистые линии обозначают алгоритмы Винограда для лучшего и худшего случаев соответственно. Красная горизонтальная линия на уровне 1 служит для сравнения с производительностью стандартного алгоритма.

3.2.3 Анализ результатов

Из проведённых замеров видно, что алгоритм Винограда (особенно его оптимизированная версия) показывает значительное улучшение времени выполнения по сравнению со стандартным алгоритмом.

При увеличении размера матриц прирост производительности становится всё более заметным. В худшем случае (нечётные размеры) время работы увеличивается, но закономерности сохраняются.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были реализованы и протестированы три алгоритма умножения матриц.

Проведённые исследования показали следующее.

— Базовый алгоритм Винограда эффективнее стандартного алгоритма умножения матриц по временной эффективности.

— Оптимизированная версия алгоритма Винограда значительно превосходит базовую по по временной эффективности.

— При увеличении размера матриц эффективность оптимизированного алгоритма становится особенно заметной.

Полученные результаты подтверждают целесообразность применения оптимизаций для алгоритмов обработки больших объёмов данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ульянов М.В. Ресурсно-эффективные компьютерные алгоритмы. — М.: ФИЗМАТЛИТ, 2008. — 304 с.

2. Джеффри Макконнелл. Анализ алгоритмов. Активный обучающий подход. — М.: Техносфера, 2009. — 416 с.

3. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — М.: Вильямс, 2015. — 1328 с.

4. Седжвик Р. Алгоритмы на C++. — М.: Вильямс, 2019. — 1056 с.

5. Кнут Д.Э. Искусство программирования. Том 3. Сортировка и поиск. — М.: Диалектика-Вильямс, 2019. — 832 с.

6. QueryPerformanceCounter function / Microsoft [эл. ресурс]. Режим доступа: `https://learn.microsoft.com/ru-ru/windows/win32/api/profileapi/nf-profileapi-queryperformancecounter` (дата обращения: 22.03.2025).